

ARRAY OF declares an array where all the items in the array are of the same fundamental type, or taken from the same set of possible types. This form of array is called a *uniform array*, and is generally used to represent ordered collections of values.

ARRAY declares an array where the types of the array items follow a particular pattern. In this form, known as a *pattern array*; the allowed type for an item depends on its position in the array. The overall pattern of types allowed in the array is declared using a schema construct called a linear type pattern, which is similar to a regular expression (see below). Pattern arrays are typically used to represent vectors, tuples or paths.

A **length** qualifier on an array MAY be used to constraint the minimum and maximum number of items in the array. For a pattern array, the given length constraint SHALL be consistent with (i.e. fall within) the minimum and maximum number of items implied by the type pattern. In cases where the **length** qualifier places a narrower constraint on the length of an array than that implied by the type pattern, the **length** qualifier constraint takes precedence.

A **nullable** qualifier MAY be used to indicate that a TLV **Null** MAY be encoded in place of the **ARRAY** or **ARRAY OF**. Note that an array that has been replaced by a Null is distinct in terms of its encoding from an array that has no items.

B.3.1.1. Linear Type Patterns

A linear type pattern describes the sequence of TLV types that MAY appear in a TLV Array or List element. In its simplest form, a linear type pattern is a list of type definitions, or references to defined types, where each item constrains the TLV type that appears at the corresponding position in the collection. The type pattern is always anchored at the start of the collection, with the first type constraining the first item in the collection. Any type or pseudo-type MAY appear within a linear type pattern.

More complex type patterns can be created by using a quantifier. Quantifiers appear after a type in a type pattern and specify the number of times the associated type MAY appear at that position in the collection. Quantifiers borrow common regular expression notation to denote repetition, with ***** meaning zero or more, **+** meaning one or more, and **{ }** expressing specific counts. Using quantifiers, one can express complex sequences of types, including some that require arbitrary look-ahead to match.

B.3.1.2. Item Names

Items or groups of items in a pattern array MAY be given textual names. These names do not affect the encoding of the array, but serve as user documentation, or as input to code generation tools. Item names within a pattern array SHALL be unique.

Per the rules for encoding TLV arrays, array items SHALL NOT have tags. Thus the tag qualifier SHALL NOT be applied to an item name with a pattern array.

ARRAY Example

```
supported-country-codes => ARRAY [ length 0..10 ] OF STRING [ length 2 ]
```